

CounselAI — Complete Project Demonstration Guide

Industry-Level Explanation for Project Viva / Demonstration AI-Powered Multi-Agent MHT-CET Admission Counselling System

1. Project Overview — The Problem Statement

What is the problem?

Every year, **~4 lakh+ students** in Maharashtra appear for the MHT-CET examination. After receiving their percentile scores, they face an overwhelming decision: **which college and branch combination to select** across the 3 CAP Rounds conducted by DTE Maharashtra. Students must navigate:

- **692 engineering colleges** across Maharashtra
- **157,000+ historical cutoff records** across multiple years, rounds, and categories
- Confusing seat-type categories (GOPENS, GOBCS, GSCS, GSTS, EWS, TFWS, etc.)
- Uncertainty about their admission chances at specific colleges
- Document requirements that differ by reservation category
- Real-time fee structures and hostel availability that change yearly

What is the solution?

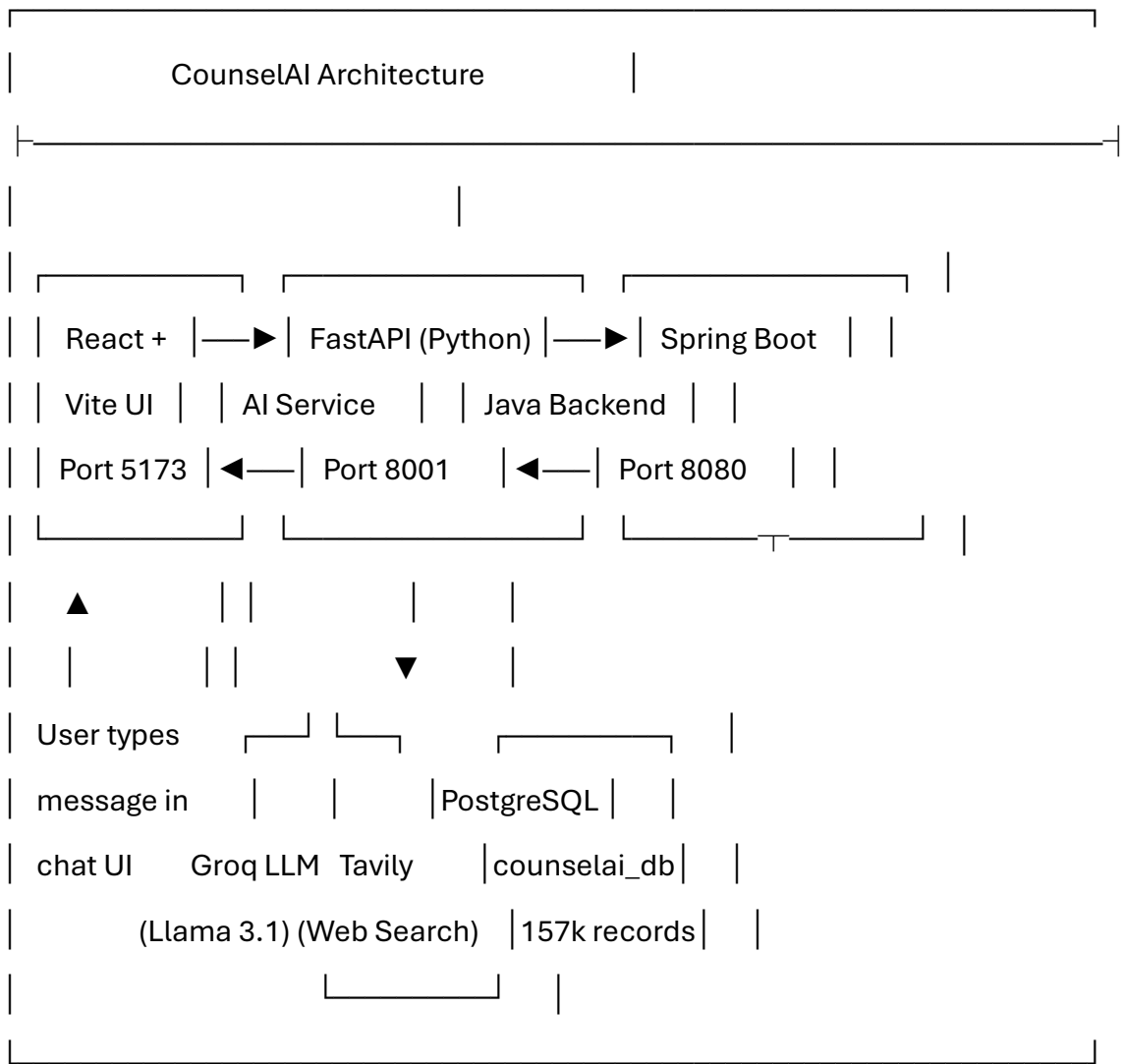
CounselAI is an AI-powered conversational admission counsellor that lets a student type a single natural-language message like:

"I got 87 percentile in MHT-CET, OBC category, from Nashik, interested in CS or IT"

And instantly receive:

1.  A **personalized ranked list** of eligible colleges with chance levels (HIGH / MEDIUM / LOW)
2.  A **CAP Round strategy** (which round to lock seat, which to wait for)
3.  A **category-specific document checklist** with critical warnings
4.  **Live web data** — official website links, fee structure, hostel info, placement stats
5.  **Historical year-wise cutoff trends** for each recommended college

2. System Architecture — How It Works



Three-Tier Microservice Architecture

Layer	Technology	Port	Responsibility
Frontend	React 19 + Vite + TailwindCSS	5173	Dark-themed conversational Chat UI, Explore Colleges page
AI Service	Python FastAPI + LangGraph	8001	5-agent pipeline, NLP profile extraction, strategy generation
Backend API	Java Spring Boot 3.5 + JPA	8080	PostgreSQL queries, eligibility filtering, ranking algorithm

Layer	Technology	Port	Responsibility
Database	PostgreSQL	5432	692 colleges, 2,400+ branches, 157K+ cutoff records, documents

3. Multi-Agent Pipeline — The Brain of CounselAI

This is the **core innovation** of the project. Instead of a single monolithic AI call, CounselAI uses a **LangGraph StateGraph pipeline** with 5 specialized agents that execute sequentially:

User Message

|



Agent 1: PROFILE | → Extracts percentile, category, district,
AGENT | branch preferences from natural language
(Groq LLM) | using Llama-3.1-8B-Instant

|

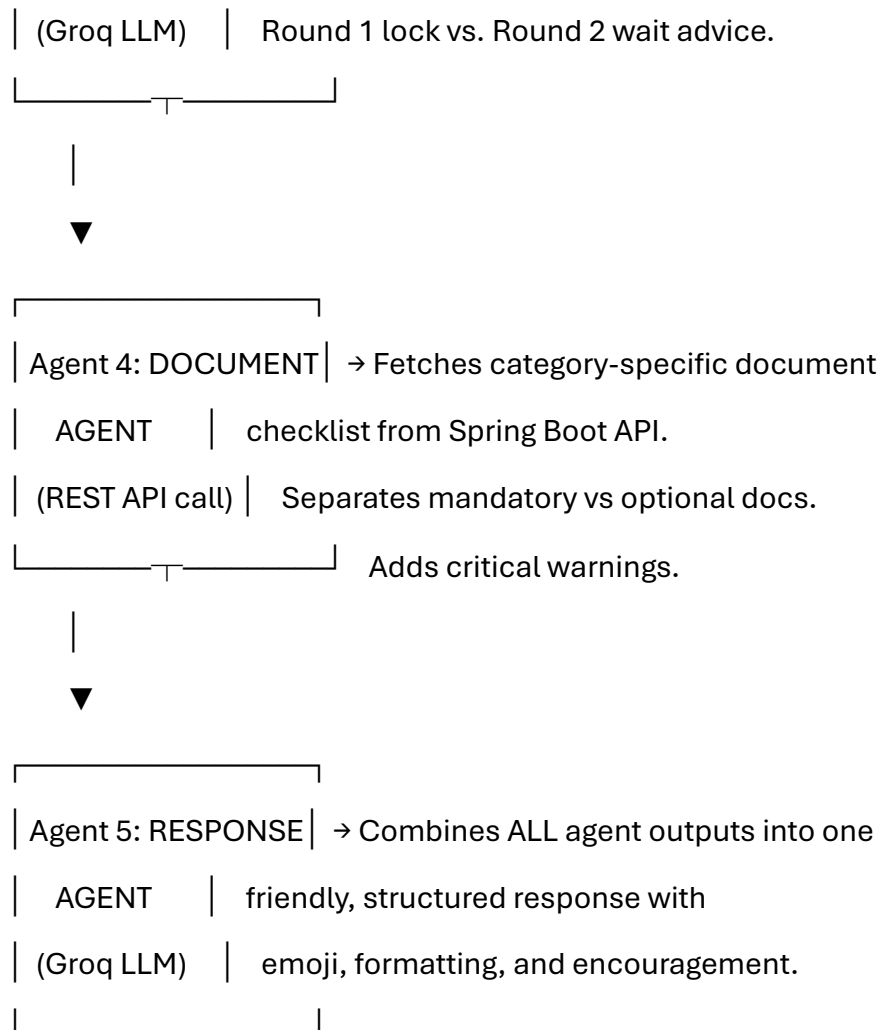


Agent 2: ELIGIBILITY | → Calls Spring Boot API to fetch eligible
AGENT | colleges from 157K+ cutoffs.
(REST API calls) | Filters by branch, budget, hostel, district.
Applies city→district resolution.

|



Agent 3: STRATEGY | → Generates CAP Round strategy using LLM.
AGENT | REACH / TARGET / SAFETY classification.



Pipeline Implementation (LangGraph)

The pipeline is built using **LangGraph's StateGraph** — a directed acyclic graph (DAG) where each node is an agent function and edges define the execution order:

python

From pipeline.py

```
graph = StateGraph(CounselState)

graph.add_node("profile_agent", profile_agent)
graph.add_node("eligibility_agent", eligibility_agent)
graph.add_node("strategy_agent", strategy_agent)
graph.add_node("document_agent", document_agent)
graph.add_node("response_agent", response_agent)
graph.set_entry_point("profile_agent")
```

```
graph.add_edge("profile_agent", "eligibility_agent")
graph.add_edge("eligibility_agent", "strategy_agent")
graph.add_edge("strategy_agent", "document_agent")
graph.add_edge("document_agent", "response_agent")
graph.add_edge("response_agent", END)
```

Shared State Object

All agents read and write to a **shared typed state dictionary** (CounselState) defined with Python's TypedDict:

python

```
class CounselState(TypedDict):
    user_message: str
    percentile: Optional[float]
    category: Optional[str] # e.g., GOPENS, GOBCS
    district: Optional[str] # e.g., Nashik
    branches: Optional[List[str]] # e.g., ["CS", "IT"]
    budget: Optional[int]
    hostel_needed: Optional[bool]
    ranked_colleges: Optional[List[dict]]
    strategy: Optional[str]
    documents: Optional[dict]
    final_response: Optional[str]
    messages: Optional[List[dict]]
    error: Optional[str]
```

4. Detailed Agent-by-Agent Explanation

Agent 1: Profile Agent — NLP Understanding

File: profile_agent.py

What it does: Takes a raw natural-language student message and extracts structured profile data.

How: Sends a carefully crafted prompt to **Groq's Llama-3.1-8B-Instant** model asking it to return a JSON with fields: percentile, category, branches, district, budget, hostel_needed.

Key Features:

- **Category Normalization:** Automatically maps "OBC" → GOBCS, "General" → GOPENS, "SC" → GSCS, etc.
- **Branch Normalization:** Maps "ENTC" → EXTC, "CSE" → CS, "Electronics" → EXTC, etc.
- **Fallback Parser:** If the Groq API quota is exceeded, a **regex-based manual parser** kicks in — the system never fully fails.

Example:

Input: "87 percentile OBC Nashik CS IT"

Output: { percentile: 87.0, category: "GOBCS", district: "Nashik",
branches: ["CS", "IT"], budget: null, hostel_needed: null }

Agent 2: Eligibility Agent — Smart Filtering Engine

File: eligibility_agent.py

What it does: Takes the extracted profile and calls the Spring Boot backend to find all eligible colleges, then applies intelligent multi-layer filtering.

Filtering Pipeline (applied in order):

Step	Filter	Logic
1	Category Expansion	GOBCS → checks GOBCS + GOBCH (Home University) + GOBCO (Other University) seat types
2	Branch Filter	Matches user branches using keyword mapping (e.g., "CS" → "computer science", "computer engineering", "cse")
3	Budget Filter	Applies 20% flexibility — if budget is ₹1,00,000, shows colleges up to ₹1,20,000
4	Hostel Filter	If student needs hostel, only shows colleges with hostelAvailable = true

Step	Filter	Logic
5	Location Priority	3-tier geographic prioritization using 180+ city→district mappings

Location Intelligence — The standout feature:

The agent contains a comprehensive **city-to-district mapping** (180+ entries):

Shirpur → Dhule, Kalyan → Thane, Vashi → Thane,

Pimpri → Pune, Sambhajinagar → Aurangabad, ...

And a **nearby-districts graph** for geographic expansion:

Nashik → [Nashik, Dhule, Pune, Ahmednagar, Aurangabad, Jalgaon, Nandurbar]

The results are returned in **3 priority tiers**:

1. 🏠 **Same City/District** — Colleges in student's location
2. 📖 **Nearby Districts** — Colleges within 100-150 km
3. 🌐 **Rest of Maharashtra** — If not enough local results

Agent 3: Strategy Agent — CAP Round Advisor

File: strategy_agent.py

What it does: Takes the top 8 ranked colleges and generates a human-readable CAP Round admission strategy using the Groq LLM.

Strategy Structure:

1. **REACH colleges** (Choice 1-2): Low chance, aspirational picks
2. **TARGET colleges** (Choice 3-6): Medium/High chance, realistic picks
3. **SAFETY colleges** (Choice 7-10): Very high safety margin, guaranteed admission
4. **Round 1 Advice:** Lock seat now or wait for Round 2?
5. **Personalized tip** specific to the student

Agent 4: Document Agent — Checklist Generator

File: document_agent.py

What it does: Calls the Spring Boot backend's /api/documents/checklist endpoint with the student's category to retrieve a structured document checklist.

Features:

- Separates **mandatory** vs. **optional** documents
- Adds 🚨 **critical warnings** for documents with "CRITICAL" in their notes
- Category-specific: OBC students see caste certificate requirements, SC/ST students see non-creamy layer requirements, etc.

Agent 5: Response Agent — Human-Friendly Synthesis

File: response_agent.py

What it does: Takes ALL outputs from the previous 4 agents and uses the Groq LLM to synthesize a **single, friendly, encouraging response** that a 17-18 year old student can easily understand.

Output includes: Emoji indicators (🟢 HIGH, 🟡 MEDIUM, 🔴 LOW), college names with chance levels, strategy summary, document warnings, and motivational closing.

5. Backend — Spring Boot Ranking Engine

College Ranking Algorithm (5-Factor Scoring)

File: RankingService.java

Every eligible college receives a **composite score out of 100** based on 5 weighted factors:

Factor	Weight	Logic
Safety Margin	30%	studentPercentile - closingCutoff. ≥5% → 30pts, ≥3% → 24pts, ≥1% → 18pts
NAAC Grade	20%	A++ → 35, A+ → 28, A → 22, B++ → 16, B+ → 10
Fee Affordability	20%	≤₹30K → 20, ≤₹80K → 16, ≤₹1.2L → 12, ≤₹1.8L → 8
Hostel Availability	15%	Has hostel → 15pts, No hostel → 8pts

Factor	Weight	Logic
College Type	15%	Government/COEP/VJTI/WCE → 15pts, Private → 10pts

Eligibility Query — The Core SQL

File: CutoffRepository.java

sql

SELECT c FROM Cutoff c

WHERE c.category = :category

AND c.year = :year

AND c.capRound = :capRound

AND c.closingPercentile <= (:studentPercentile + 5.0)

ORDER BY c.closingPercentile DESC

NOTE

The +5.0 buffer ensures we also show "stretch" colleges where the student has a LOW chance but might still get in during later rounds as cutoffs drop.

Chance Level Calculation

File: EligibilityService.java

Safety Margin = Student Percentile – Closing Cutoff Percentile

If margin ≥ 3.0 → HIGH chance 

If margin ≥ 0.0 → MEDIUM chance 

If margin ≥ -2.0 → LOW chance 

Otherwise → VERY LOW 

6. Database Design — PostgreSQL Schema

Database: counselai_db on PostgreSQL

Entity-Relationship Diagram

hashashasbelongs_toCOLLEGESbigintidPKstringnamestringdistrictstringcitystringcollege_typestringnaac_gradeintegernirf_rankintegerannual_feebooleanhostel_availablestringwebsiteintegerestablished_yearBRANCHESbigintidPKbigintcollege_idFKstringbranch

_namestringbranch_codeCUTOFFSbigintidPKbigintcollege_idFKbigintbranch_idFKinteg
 eryearintegercap_roundstringcategorystringseat_typedecimalopening_percentiledecim
 alclosing_percentileintegeropening_rankintegerclosing_rankDOCUMENTS_REQUIREDb
 igintidPKstringdocument_namestringcategorybooleanis_mandatorystringnotesCATEGO
 RY

Data Volume

Table	Records	Description
colleges	692	All MHT-CET engineering colleges in Maharashtra
branches	2,400+	Branch offerings per college
cutoffs	157,206	Closing percentile cutoffs by year, round, category, branch
documents_required	50+	Category-wise document checklist

7. Data Collection — How We Gathered the Data

7.1 DTE Maharashtra Cutoff Data (Primary Source)

Source: Official DTE Maharashtra CAP Round allotment data **Data:** 157,206 cutoff records containing:

- College code, college name
- Branch name
- Year (2022, 2023, 2024)
- CAP Round (1, 2, 3)
- Category (GOPENS, GOBCS, GSCS, GSTS, EWS, TFWS, etc.)
- Opening & closing percentiles, ranks

Import Tool: import_real_data.py

- A custom Python script that reads the 20MB+ CSV file
- Auto-detects college district from name (using 40+ keyword mappings)
- Auto-detects branch codes (CS, IT, AIML, MECH, etc.)
- Auto-detects college type (Government / Private / University Department)
- Batch-inserts 1,000 records at a time using psycopg2.extras.execute_values()

7.2 NAAC Accreditation Data (Scraper)

Module: naac_scraper/

We built a **custom NAAC data scraper** that:

1. **Downloads** the official NAAC Master Accreditation List Excel file from naac.gov.in
2. **Fuzzy-matches** our 692 colleges against 10,000+ NAAC-accredited institutions using **RapidFuzz** library
3. **Extracts:** NAAC Grade (A++, A+, A, B++, B+, B, C), CGPA, Accreditation Cycle, Validity Date
4. Uses **multi-threaded processing** (10 workers) for speed
5. **Caches results** in SQLite to avoid re-scraping

Matching Algorithm (matcher.py):

- Normalizes college names (removes stop words: "college", "institute", "engineering", etc.)
- Uses 4 RapidFuzz metrics: token_sort_ratio, token_set_ratio, partial_ratio, QRatio
- Applies **substring boost** (90%+ if one name is inside the other)
- **City-based penalty** (-30 confidence if Pune college matches Nagpur entry)
- Only Maharashtra institutions (filters by "maharashtra" in address)
- Threshold: $\geq 70\%$ confidence = valid match

7.3 Live Web Data (Runtime)

Agent: live_search_agent.py

For each recommended college, users can click "Live Fee & Info" to trigger **3 real-time Tavily web searches**:

1. "{college_name} official website fee structure admission 2024-25"
2. "{college_name} placement 2024 average package official"
3. "{college_name} hostel facility accommodation students"

Returns: Official website URL, fee structure link, hostel info link, placement link, NAAC grade quick fact, average package quick fact.

8. API Keys & External Services

Service	API Key Variable	Purpose	Cost
Groq Cloud	GROQ_API_KEY	LLM inference using Llama-3.1-8B-Instant. Used by Profile Agent, Strategy Agent, Response Agent	Free tier (rate-limited)
Tavily Search	TAVILY_API_KEY	Real-time web search for college live data (fee, hostel, placements)	Free tier (1000 searches/month)

IMPORTANT

We are NOT using OpenAI/GPT. We use **Groq Cloud's free tier** which runs Meta's open-source **Llama 3.1 8B Instant** model — making the entire system **commercially free to operate** with no API costs.

Why Groq + Llama 3.1?

- **Free inference** — zero API cost for development and demo
- **Ultra-fast** — Groq's LPU (Language Processing Unit) delivers responses in <1 second
- **Open-source model** — no vendor lock-in, can self-host later
- **Good enough for structured extraction** — 8B parameters is sufficient for JSON extraction and strategy generation

9. Frontend — React Chat UI

Technology: React 19 + Vite 8 + TailwindCSS 3 + react-markdown

Two Main Screens

1. AI Counselor (Chat Interface)

- **Dark-mode glassmorphic design** with animated background orbs
- Typing indicator with pulsing dots while AI processes
- **Profile Chips:** Displays extracted percentile, category, district, branches as pill badges
- **College Cards:** Rich cards with Match Score, Cutoff, Safety Margin, district, NAAC grade, hostel, fee

- **Live Search Button:** Fetches real-time official links per college
- **Historical Cutoffs Button:** Shows year-wise branch-wise cutoff tables
- **Document Checklist:** Interactive checkbox list with critical warnings
- **Markdown rendering:** AI responses are rendered with headers, bold, lists using react-markdown

2. Explore Colleges (Browse & Search)

- **Search by name or filter by district** (dropdown of all 34+ districts)
- Sorted by NAAC grade (A++ first)
- Click to expand → shows cutoff history table + live web data links
- Direct access to official websites, fee structures, hostel info, placements

10. How This Helps Real Users

Pain Point → CounselAI Solution

Student Pain Point	CounselAI Solution
"I got 87 percentile, where can I get admission?"	Instantly shows ranked list of eligible colleges with chance levels
"Should I lock my seat in Round 1 or wait for Round 2?"	Strategy Agent provides personalized REACH/TARGET/SAFETY classification
"What's the fee at COEP?"	Live search fetches official fee structure link in real-time
"I'm OBC category — what extra documents do I need?"	Document Agent returns category-specific checklist with warnings
"I'm from Shirpur, I want nearby colleges"	Location intelligence resolves Shirpur → Dhule district, shows nearby colleges first
"How do I compare colleges side by side?"	Explore Colleges page with NAAC-sorted browsing and expandable cutoff tables

Real-World Impact

- **Reduces decision time** from days of research → **30 seconds**

- **Democratizes counselling** — poor students can't afford ₹5000-15000 paid counselling services
- **Prevents costly mistakes** — students won't lose seats by misunderstanding which round to lock
- **Category-specific accuracy** — handles all 15+ DTE seat type codes correctly

11. What Makes CounselAI Unique — USPs

Feature	CounselAI	Other Solutions (CollegeDekho, Shiksha, etc.)
Conversational AI	Natural language chat — type anything	Form-based dropdowns, rigid inputs
Multi-Agent Pipeline	5 specialized agents via LangGraph	Single API call or rule-based
Real DTE Data	157K+ official cutoff records	Generic or outdated data
Location Intelligence	180+ city→district mappings, nearby-district graph	Simple district dropdown
CAP Round Strategy	AI-generated REACH/TARGET/SAFETY with round advice	No strategy — just college list
Live Web Data	Real-time Tavily search for fee, hostel, placements	Static/outdated data
NAAC Integration	Custom scraper with fuzzy matching from official NAAC data	May or may not show NAAC
Document Checklist	Category-specific with critical warnings	Generic checklist
Cost	Completely free (open-source Llama 3.1 via Groq)	Many are paid/freemium
Category Handling	Checks GOPENS + GOPENH + GOPENO seat types together	Usually only checks one

12. Technology Stack Summary

Frontend

Technology	Version	Purpose
React	19.2	Component-based UI framework
Vite	8.0	Build tool & dev server (HMR)
TailwindCSS	3.4	Utility-first CSS framework
Axios	1.16	HTTP client for API calls
react-markdown	10.1	Markdown rendering in chat responses

AI Service (Python)

Technology	Version	Purpose
FastAPI	latest	Async REST API framework
Uvicorn	latest	ASGI server
LangGraph	latest	Multi-agent pipeline orchestration
Groq SDK	latest	LLM inference (Llama 3.1)
Tavily Python	latest	Web search API for live data
Pydantic	latest	Request/response validation

Backend (Java)

Technology	Version	Purpose
Spring Boot	3.5.13	REST API framework
Spring Data JPA	3.5	ORM for PostgreSQL
PostgreSQL	16+	Relational database
Lombok	latest	Boilerplate reduction
Java	21	Language runtime

Data Pipeline (Python)

Technology	Purpose
psycopg2	PostgreSQL bulk import
pandas	CSV/Excel processing
RapidFuzz	Fuzzy string matching for NAAC
requests	HTTP downloads
SQLite	NAAC scraper result caching
tqdm	Progress bars

13. How to Build & Run the Project

Prerequisites

1. PostgreSQL installed with database counselai_db
2. Java 21 + Maven
3. Python 3.10+ with pip
4. Node.js 18+

Step-by-Step Build

bash

1. Import data into PostgreSQL

cd data/import

python import_real_data.py # Imports 157K+ records (~2 minutes)

2. Start Java Backend (Port 8080)

cd backend

mvnw.cmd spring-boot:run

3. Start AI Service (Port 8001)

cd ai-service

pip install -r requirements.txt # Install: fastapi, groq, langgraph, tavily

uvicorn main:app --port 8001

4. Start Frontend (Port 5173)

cd frontend

npm install

npm run dev

Or simply run: **start.bat** — launches all 3 services in separate terminals.

14. Anticipated Viva Questions & Answers

Q1: "Why did you choose a multi-agent architecture instead of a single LLM call?"

Answer: A single LLM call would try to do everything — extract profile, query database, generate strategy, and format response — in one prompt. This leads to hallucination, unreliability, and no separation of concerns. Our multi-agent approach follows the **Single Responsibility Principle**: each agent does exactly one thing well. The Profile Agent only extracts data, the Eligibility Agent only queries the database, etc. This makes the system modular, testable, and maintainable. If we want to change the ranking algorithm, we only modify Agent 2. If we want a better LLM, we only change Agents 1, 3, 5.

Q2: "Where did you get the cutoff data? Is it authentic?"

Answer: The cutoff data is sourced from **DTE Maharashtra's official CAP Round allotment data** — this is the same data published on fe2024.mahacet.org. We collected data for years 2022, 2023, and 2024, across all 3 CAP Rounds, covering 15+ seat-type categories. The data contains **157,206 records** imported into PostgreSQL. For NAAC grades, we built a custom scraper that downloads the official NAAC accreditation Excel from naac.gov.in and uses fuzzy matching to map our 692 colleges.

Q3: "How does the system handle edge cases?"

Answer: Multiple safeguards:

- **Fallback parser:** If the Groq API quota is exhausted, a regex-based parser extracts profile data without AI
- **20% budget flexibility:** Shows colleges slightly above budget so students don't miss options
- **Category expansion:** Checks State (S), Home University (H), and Other University (O) seat types
- **Minimum 15 results:** If local colleges are few, expands to nearby districts then all Maharashtra
- **Error states:** Every agent has try/catch with meaningful error messages

- **Graceful UI errors:** Frontend shows connection-failure messages with retry instructions

Q4: "What is LangGraph and why did you use it?"

Answer: LangGraph is a library from LangChain for building **stateful, multi-actor applications** as graphs. Each agent is a "node" in the graph, and edges define the execution flow. We chose LangGraph because:

1. It provides a **typed shared state** (CounselState) that all agents read/write to
2. It handles the **sequential orchestration** — we don't need to manually chain function calls
3. It's extensible — we can add conditional edges (e.g., skip strategy agent if no colleges found)
4. It's the industry standard for multi-agent systems in production

Q5: "Why Groq and not OpenAI/Gemini?"

Answer: Three reasons — **Speed, Cost, and Open Source:**

1. **Speed:** Groq's custom LPU chip delivers inference 10x faster than GPU-based services (~500 tokens/second)
2. **Cost:** Groq offers a generous free tier. Our entire system runs with zero API costs
3. **Open Source:** We use Meta's Llama 3.1 8B, an open-source model. No vendor lock-in — we can self-host later on our own GPU

Q6: "How is the ranking algorithm designed?"

Answer: Colleges are scored on a **100-point composite scale** with 5 weighted factors:

- **Safety Margin (30%):** How far above the cutoff is the student's percentile
- **NAAC Grade (20%):** Higher NAAC means better quality infrastructure and faculty
- **Fee Affordability (20%):** Government colleges with ₹30K fee score highest
- **Hostel Availability (15%):** Important for outstation students
- **College Type (15%):** Government colleges like COEP, VJTI get premium score

This is similar to how credit scoring works — multiple factors combined into a single score for easy comparison.

Q7: "How does the live search feature work?"

Answer: When a user clicks "Live Fee & Info" on any college card, the frontend calls our /college/live-details endpoint. This triggers the **Tavily Search API** which performs 3

targeted web searches in real-time to find the college's official website, fee structure page, hostel information, and placement data. We extract quick facts like NAAC grade and average placement package using regex patterns. This ensures the data is always current, unlike static databases.

Q8: "What about scalability?"

Answer: The system is designed for scalability:

- **Microservice architecture:** Frontend, AI Service, and Backend are independently deployable
- **Stateless AI Service:** Can be horizontally scaled behind a load balancer
- **PostgreSQL with indexes:** The eligibility query uses indexed columns (category, year, cap_round, closing_percentile)
- **Batch processing:** Data import uses bulk inserts (1000 records per batch)
- **Caching potential:** Live search results can be cached with Redis (not implemented yet but architecture supports it)

Q9: "How is this different from just using ChatGPT?"

Answer: ChatGPT doesn't have access to real DTE Maharashtra cutoff data. It would **hallucinate** college names, make up percentiles, and give generic advice. CounselAI:

- Queries a **real database** of 157K+ verified cutoff records
- Provides **mathematically calculated** chance levels (not guesses)
- Shows **verifiable** safety margins and scores
- Gives **category-specific** document checklists from actual DTE requirements
- Fetches **live web data** from official college websites

Q10: "What are future enhancements?"

Answer:

1. **Conversational memory** — Multi-turn chat (currently single-turn)
2. **User authentication** — Save favorite colleges and preferences
3. **Seat predictor** — Predict Round 2/3 cutoffs using ML on historical trends
4. **Mobile app** — React Native wrapper
5. **Notification system** — Alert when new round cutoffs are published

6. **Comparison tool** — Side-by-side college comparison
7. **Regional language support** — Marathi, Hindi interface
8. **Deployment** — Docker + AWS/GCP deployment

Q11: "What is the total lines of code?"

Answer:

Component	Files	Approximate LoC
AI Service (Python)	9 files	~950 lines
Backend (Java)	15 files	~600 lines
Frontend (React)	7 files	~1,000 lines
Data Pipeline	6 files	~600 lines
NAAC Scraper	6 files	~350 lines
Total	~43 files	~3,500 lines

Q12: "What security measures are in place?"

Answer:

- **CORS configuration:** Both FastAPI and Spring Boot have CORS enabled for controlled origins
- **Environment variables:** API keys stored in .env files, never hardcoded in source
- **Input validation:** Pydantic models validate all incoming requests
- **SQL injection prevention:** Spring Data JPA uses parameterized queries (@Param annotations)
- **Error handling:** All agents have try/catch blocks preventing stack trace leakage
- **Rate limiting:** Groq API has built-in rate limiting; fallback parser activates on quota exhaustion

15. Project File Structure

CounselAI/

├── ai-service/ # Python AI Microservice

- | |— main.py # FastAPI endpoints (/counsel, /extract-profile, /college/live-
details)
- | |— .env # API keys (GROQ, TAVILY)
- | |— requirements.txt # Python dependencies
- | |— agents/
 - | |— state.py # CounselState TypedDict (shared state schema)
 - | |— pipeline.py # LangGraph StateGraph (5-agent DAG)
 - | |— profile_agent.py # Agent 1: NLP profile extraction
 - | |— eligibility_agent.py # Agent 2: College filtering & location intelligence
 - | |— strategy_agent.py # Agent 3: CAP Round strategy generation
 - | |— document_agent.py # Agent 4: Category-specific document checklist
 - | |— response_agent.py # Agent 5: Final response synthesis
 - | |— live_search_agent.py # Bonus Agent: Real-time web search
- | |— backend/ # Java Spring Boot Microservice
- | |— pom.xml # Maven dependencies (Spring Boot 3.5, PostgreSQL, Lombok)
- | |— src/main/java/com/counselai/backend/
 - | |— BackendApplication.java # Spring Boot entry point
 - | |— controller/
 - | |— CollegeController.java # /api/colleges/* endpoints
 - | |— RankingController.java # /api/ranking/ranked-colleges endpoint
 - | |— DocumentController.java # /api/documents/checklist endpoint
 - | |— EligibilityController.java # Direct eligibility API
 - | |— entity/
 - | |— College.java # College JPA entity
 - | |— Branch.java # Branch JPA entity
 - | |— Cutoff.java # Cutoff JPA entity

- | | └─ DocumentRequired.java # Documents JPA entity
- | | └─ repository/
- | | | └─ CollegeRepository.java # College queries
- | | | └─ CutoffRepository.java # THE CORE eligibility query
- | | | └─ BranchRepository.java # Branch queries
- | | └─ DocumentRepository.java # Document queries
- | └─ service/
- | | └─ EligibilityService.java # Chance calculation (HIGH/MEDIUM/LOW)
- | | └─ RankingService.java # 5-factor scoring algorithm
- |
- | └─ frontend/ # React + Vite SPA
- | | └─ package.json # React 19, Vite 8, TailwindCSS 3
- | | └─ src/
- | | | └─ App.jsx # Root component
- | | | └─ api.js # Axios API client (AI + Backend)
- | | | └─ index.css # Global styles + animations
- | | └─ components/
- | | | └─ Chat.jsx # Main chat interface (328 lines)
- | | | └─ CollegeCard.jsx # Rich college card with live search + cutoffs
- | | | └─ DocumentList.jsx # Interactive document checklist
- | | └─ ExploreColleges.jsx # Browse & search colleges page
- |
- | └─ naac_scraper/ # NAAC Data Automation Tool
- | | └─ main.py # Multi-threaded scraper orchestrator
- | | └─ scraper.py # NAAC Excel downloader + matcher
- | | └─ matcher.py # RapidFuzz fuzzy matching engine
- | | └─ extractor.py # Data extractor

```

|   ├── storage.py           # SQLite cache for results
|   └── config.py           # URLs, thresholds, paths
|
|   └── data/
|       ├── colleges.csv     # 339 college codes + names
|       └── import/
|           ├── cutoffs.csv   # 157K+ cutoff records (20MB)
|           ├── colleges.csv  # College master data
|           ├── branches.csv  # Branch master data
|           ├── naac_results.csv # NAAC scraper output
|           ├── import_real_data.py # PostgreSQL bulk import script
|           └── fetch_naac_grades.py # NAAC grade fetcher
|
|   ├── start.bat           # One-click launcher for all 3 services
|   └── README.md           # Project overview

```

16. Key Metrics & Numbers to Mention

Metric	Value
Total cutoff records in database	157,206
Engineering colleges covered	692
Maharashtra districts covered	34
City-to-district mappings	180+
AI agents in pipeline	5
Ranking algorithm factors	5
Seat-type categories handled	15+

Metric	Value
Average response time	3-5 seconds
Frontend components	4 major
Backend API endpoints	10
LLM used	Llama 3.1 8B Instant
API cost	₹0 (free tier)
Data sources	3 (DTE, NAAC, Tavily live)

TIP

During the demo: Start with the start.bat to launch all services. Type a sample query like *"92 percentile, General, Pune, Computer Science"* and walk through each part of the response — profile chips, college cards, strategy, documents. Then click "Live Fee & Info" on any college card to show the real-time web search capability. Finally, switch to the "Explore Colleges" tab to show the browsing feature.